

PRD: Pizdit?

1. Product Summary

Pizdit? is a global, open-source civic-tech application that helps voters understand upcoming elections in their area, see who will appear on their ballot, and evaluate whether politicians' claims, promises, and public statements are supported by evidence.

The core user experience:

Open the app → enter an address → see upcoming elections → see candidates → understand which claims are supported, unsupported, misleading, or unverifiable.

The product must not tell users who to vote for. It must provide structured, evidence-based analysis, sources, risk indicators, and audit trails.

Pizdit? should feel extremely simple on the surface and deeply rigorous underneath.

2. Product Vision

Pizdit? should become a global open-source infrastructure for political claim verification.

The long-term goal is to make candidate analysis available for any voter in any country where enough public election and candidate data exists.

The product should support:

- local elections;
- regional elections;
- national elections;
- presidential elections;
- parliamentary elections;
- referendums;
- party-list systems;
- candidate-based systems;
- mixed electoral systems.

The app should be built as a global platform with country-specific data adapters, localization, jurisdiction-aware analysis, and transparent methodology.

3. Product Positioning

Public-facing tone

The brand can be provocative, simple, direct, and memorable.

Example:

```text id="a9gdx5"

Pizdit?

Check what politicians are saying before you vote.

Stronger alternative:

```text id="kq031w" Find out who talks bullshit before election day.

Analytical tone

The internal analytical layer must be careful, neutral, evidence-based, and legally safer.

The system should avoid directly calling a politician a liar.

Do not write:

```text id="u2ssu0"

This candidate is lying.

Prefer:

```text id="vl5byf" This specific claim is not supported by the available evidence.

Do not write:

```text id="i1rb1o"

This candidate is corrupt.

Prefer:

```text id="q5glen" Available public records show unresolved allegations reported by named sources. The system has not independently verified criminal wrongdoing.

The brand can be sharp. The analysis must be rigorous.

4. Product Principles

4.1 No registration for public users

Public users must not be required to create an account.

No email login.

No password.

No OAuth.

No public user profile.

No onboarding wall.

The user should be able to open the app and immediately use it.

4.2 Address-first experience

The first meaningful user action is entering an address or selecting a region manually.

The app uses the address only to determine:

- country;
- region;
- municipality;
- electoral district;
- relevant upcoming elections;
- possible ballot candidates or parties.

4.3 Global-first architecture

The product must not be designed around a single region.

It should support global coverage through country-specific data adapters.

Each country may have different:

- election systems;
- ballot structures;
- official data sources;
- district boundary formats;
- party systems;
- candidate registration formats;
- campaign disclosure rules;
- languages;
- source reliability norms;
- legal risks.

The core product should be universal. Data ingestion and jurisdiction logic

```
should be modular.
```

```
---
```

4.4 Simple surface UX

The public product should be very simple.

Main public flow:

```
```text id="j5dqtw"
```

```
Address → Election → Candidates → Verdict → Evidence
```

Advanced analysis should exist, but it should be hidden behind simple cards and expandable sections.

---

### 4.5 Evidence over opinion

Every verdict must be traceable to:

- source URLs;
- checked claims;
- evidence items;
- confidence score;
- audit result;
- prompt version;
- model version;
- timestamp;
- explanation.

No black-box candidate scoring.

---

### 4.6 No direct voting recommendation

The product must not say:

```
```text id="fqc343" Vote for Candidate A.
```

It may say:

```
```text id="p1z6ns"
```

```
Candidate A has fewer unsupported claims among the statements checked by the system.
```

It may also say:

```text id="mb3iv4" Candidate B has several claims that lack budget details, legal authority, or implementation plans.

4.7 No live AI calls in voter-facing flow

Public users must not trigger live AI analysis.

The public app should only read already-processed, audited, cached data from the database.

Correct model:

```text id="umxot1"

Admin/batch pipeline checks claims → stores audited results → public user reads cached results.

Incorrect model:

```text id="n92an6" Public user opens candidate page → AI analyzes candidate from scratch.

This is important for:

- speed;
- cost control;
- reliability;
- auditability;
- legal safety;
- reproducibility.

4.8 AI must be auditable

AI output must not be published directly without audit.

The fact-checking pipeline must include:

1. Independent Research Agent A
2. Independent Research Agent B
3. Independent Audit Agent

The Audit Agent compares both research outputs and decides whether the result is publishable.

4.9 Open-source trust

The project should be open-source by default.

The repository should include:

- application code;
- data schemas;
- public methodology;
- claim rating rules;
- source quality rules;
- prompts;
- audit rules;
- candidate verdict formula;
- evaluation fixtures;
- test claims;
- example adapters;
- public documentation.

Open-source is not only a licensing decision. It is part of the trust model.

Users, journalists, researchers, and civic organizations should be able to inspect how the system works.

5. Target Users

5.1 Primary user

A normal voter who:

- does not follow politics every day;
- does not know all local candidates;
- wants a quick overview before voting;
- does not want to read long political programs;
- wants evidence, not campaign propaganda.

5.2 Secondary users

- journalists;
- civic activists;
- researchers;
- local media;
- watchdog organizations;
- independent fact-checkers;
- election transparency organizations;
- open-data contributors.

6. Global MVP Scope

The MVP should be global-first but not globally complete on day one.

The system must be technically capable of supporting the whole world, but data coverage can start with a limited number of countries, elections, or regions.

The MVP should support:

1. multiple countries in the data model;
2. multiple election systems;
3. country-specific data adapters;
4. country-specific source quality rules;
5. localization-ready UI;
6. multilingual claim storage;
7. no public authentication;
8. batch AI processing;
9. cached public results;
10. open-source methodology.

7. MVP Must Include

1. Address input
2. Manual country/region selection fallback
3. Geocoding and district matching
4. Upcoming election list for the user's district
5. Candidate or party list for the selected election
6. Candidate cards
7. Candidate detail pages
8. Claim detail pages
9. Evidence-based AI analysis
10. Three-stage AI fact-checking pipeline:
 - Research Agent A
 - Research Agent B
 - Audit Agent
11. Source list for every checked claim
12. Simple verdict labels
13. Admin panel for managing elections, candidates, sources, claims, and reviews
14. Human review queue for risky or low-confidence results
15. Country adapter interface
16. Open-source methodology directory
17. Prompt versioning
18. Model versioning
19. Traceability for every published check
20. No live AI calls in public voter-facing flow

8. MVP Must Not Include

- public user accounts;
- comments;
- social feeds;
- likes;
- public polls;
- user-generated candidate ratings;
- direct vote recommendations;
- personalized political matching;
- gamification;
- party discussion rooms;
- anonymous allegations;
- live AI analysis triggered by public users.

9. Core User Flow

9.1 First launch

1. User opens the app.
2. App shows:
 - product name;
 - one-line explanation;
 - address input;
 - manual country/region selection fallback;
 - short privacy note.
3. User enters address.
4. App geocodes the address.
5. App maps the result to an electoral district.
6. App shows upcoming elections for that district.

Example UI copy:

```
```text id="xhkako"
Enter your address to see what is on your ballot.
No account needed.
```

Privacy note:

```
```text id="7gp1sx" We use your address only to find your election district.
```

9.2 Election overview

The election overview screen shows:

- election name;
- election date;
- election level;
- country;
- region;
- municipality;
- district;
- polling place link, if available;
- list of candidates or parties.

Example:

```
```text id="e6yzp5"
```

Upcoming election

Municipal Council Election

Porto, Portugal

Date: 12 October 2026

Your ballot:

- Candidate A
- Candidate B
- Candidate C
- Party D

---

## 9.3 Candidate list

Each candidate card shows:

- candidate name;
- party or independent status;
- verdict badge;
- top risk tags;
- short summary;
- link to detailed analysis.

Example:

```
```text id="5xphk3" Anna Silva Independent
```

Pizdit? Likely yes

Main flags: - Promises without budget - Uses misleading statistics - No implementation plan

Summary: Several key promises lack funding details, timelines, and legal mechanisms.

9.4 Candidate detail

The candidate detail page includes:

1. Candidate summary
2. Overall verdict
3. Top concerns
4. Reasonable proposals
5. Checked claims
6. Program analysis
7. Public statement analysis
8. Voting record or public record analysis, if available
9. Source list
10. Audit status summary
11. Last updated timestamp

9.5 Claim detail

The claim detail page includes:

- original claim;
- source of the claim;
- date of the claim, if known;
- final rating;
- confidence score;
- short explanation;
- detailed explanation;
- evidence list;
- conflicting evidence, if any;
- Agent A/B agreement status;
- audit result;
- human review status, if applicable;
- prompt version;
- model version;
- source snapshot timestamp.

10. Verdict System

Avoid fake precision such as:

```
```text id="46m6mv"
```

This candidate is lying 73%.

Use categorical verdicts instead.

---

## 10.1 Public verdict labels

```text id="v0pxa6" Yes, often Likely yes Mixed Likely no Insufficient data

```
---  
  
## 10.2 Internal verdict enum  
  
```ts id="e1kyr0"  
type PizditVerdict =
 | "YES_OFTEN"
 | "LIKELY_YES"
 | "MIXED"
 | "LIKELY_NO"
 | "INSUFFICIENT_DATA";
```

---

## 10.3 Verdict meaning

Public label	Internal value	Meaning
Yes, often	YES_OFTEN	Many checked claims are unsupported, misleading, false, or contradicted by evidence
Likely yes	LIKELY_YES	Several important checked claims are problematic
Mixed	MIXED	The record contains both supported and problematic claims
Likely no	LIKELY_NO	Most checked claims are supported or reasonably argued
Insufficient data	INSUFFICIENT_DATA	Not enough reliable evidence to produce a stable verdict

---

## 11. Claim Rating System

Claim-level rating:

```ts id="z9o03s" type ClaimRating = | "TRUE" | "MOSTLY\_TRUE" | "MIXED" | "MISLEADING" |  
"UNSUPPORTED" | "FALSE" | "UNVERIFIABLE";

Meanings:

```
Rating	Meaning
TRUE	The claim is strongly supported by reliable evidence
```

```
| MOSTLY_TRUE | The claim is mostly supported but may miss some context |
| MIXED | The claim contains both accurate and problematic elements |
| MISLEADING | The claim uses selective framing, missing context, or
distorted interpretation |
| UNSUPPORTED | There is not enough evidence to support the claim |
| FALSE | Reliable evidence contradicts the claim |
| UNVERIFIABLE | The claim cannot be checked with available reliable sources
|
```

```
---
```

12. Analysis Tags

Candidates and claims may have multiple tags.

```
```ts id="lh8e9v"
type AnalysisTag =
 | "UNSUPPORTED_CLAIM"
 | "MISLEADING_STATISTICS"
 | "NO_BUDGET"
 | "NO_IMPLEMENTATION_PLAN"
 | "OUTSIDE_AUTHORITY"
 | "CONTRADICTS_VOTING_RECORD"
 | "POSITION_FLIP"
 | "FEAR_APPEAL"
 | "SCAPEGOATING"
 | "FALSE_DILEMMA"
 | "VAGUE_PROMISE"
 | "CONFIRMED_CLAIM"
 | "REALISTIC_PROPOSAL"
 | "CLEAR_BUDGET"
 | "CLEAR_TIMELINE"
 | "INSUFFICIENT_DATA";
```

Example public tag labels:

Internal tag	Public label
UNSUPPORTED_CLAIM	Unsupported claim
MISLEADING_STATISTICS	Misleading statistics
NO_BUDGET	No budget
NO_IMPLEMENTATION_PLAN	No implementation plan
OUTSIDE_AUTHORITY	Outside legal authority
CONTRADICTS_VOTING_RECORD	Contradicts voting record
POSITION_FLIP	Position changed
FEAR_APPEAL	Fear-based rhetoric

Internal tag	Public label
SCAPEGOATING	Scapegoating
FALSE_DILEMMA	False dilemma
VAGUE_PROMISE	Vague promise
CONFIRMED_CLAIM	Confirmed claim
REALISTIC_PROPOSAL	Realistic proposal
CLEAR_BUDGET	Clear budget
CLEAR_TIMELINE	Clear timeline
INSUFFICIENT_DATA	Insufficient data

## 13. Global Data Model

### 13.1 Country

```
``ts id="d1yrk8" type Country = { id: string; isoCode: string; name: string; defaultLanguage: string;
supportedLanguages: string[]; electionSystemNotes?: string; legalRiskNotes?: string;
officialElectionSourceUrls: string[]; adapterStatus: | "NOT_STARTED" | "PLANNED" | "IN_PROGRESS" |
"BETA" | "SUPPORTED" | "DEPRECATED"; createdAt: string; updatedAt: string; };
```

```

```

```
13.2 Election
```

```
``ts id="y6zvqv"
type Election = {
 id: string;
 countryCode: string;
 name: string;
 regionName?: string;
 municipalityName?: string;
 districtName?: string;
 electionLevel:
 | "municipal"
 | "regional"
 | "national"
 | "presidential"
 | "parliamentary"
 | "referendum"
 | "other";
 ballotType:
 | "candidate"
 | "party_list"
 | "mixed"
```

```

 | "referendum_question"
 | "unknown";
electionDate: string;
registrationDeadline?: string;
pollingPlaceUrl?: string;
sourceUrls: string[];
createdAt: string;
updatedAt: string;
};

```

## 13.3 Electoral district

```ts id="ipd2p5" type ElectoralDistrict = { id: string; countryCode: string; regionName?: string; municipalityName?: string; districtName: string; districtCode?: string; boundaryGeoJson?: unknown; sourceUrls: string[]; createdAt: string; updatedAt: string; };

```

---

## 13.4 Candidate

```ts id="q85mij"
type Candidate = {
 id: string;
 electionId: string;
 countryCode: string;
 fullName: string;
 partyName?: string;
 partyId?: string;
 isIndependent: boolean;
 photoUrl?: string;
 officialWebsiteUrl?: string;
 socialUrls?: string[];
 shortBio?: string;
 currentRole?: string;
 previousRoles?: string[];
 districtName?: string;
 sourceLanguage?: string;
 createdAt: string;
 updatedAt: string;
};

```

## 13.5 Party

```ts id="e8usge" type Party = { id: string; countryCode: string; name: string; shortName?: string; officialWebsiteUrl?: string; socialUrls?: string[]; ideologyDescription?: string; sourceUrls: string[]; createdAt: string; updatedAt: string; };

```

---

## 13.6 Candidate source

```ts id="n7q1yo"
type CandidateSource = {
 id: string;
 candidateId: string;
 url: string;
 title?: string;
 publisher?: string;
 countryCode: string;
 language?: string;
 sourceType:
 | "official_candidate"
 | "official_party"
 | "election_commission"
 | "government"
 | "public_record"
 | "statistics_database"
 | "media"
 | "fact_checking_org"
 | "ngo"
 | "academic"
 | "social_media"
 | "other";
 reliabilityLevel: "high" | "medium" | "low";
 addedAt: string;
};

```

## 13.7 Political claim

A political claim is a checkable statement made by a candidate, party, campaign, or official representative.

```

```ts id="vp45fv" type PoliticalClaim = { id: string; candidateId?: string; partyId?: string; electionId: string;
countryCode: string; claimText: string; originalLanguage: string; translatedText?: string; claimType: |
"statistical" | "promise" | "attack" | "policy_position" | "past_record" | "future_projection" | "other";
claimSourceUrl: string; claimSourceTitle?: string; claimDate?: string; context?: string; jurisdictionLevel?:
"municipal" | "regional" | "national" | "international" | "unknown"; createdAt: string; updatedAt:
string; };

```

```

---

## 13.8 Evidence item

```ts id="hwx6ii"

```

```

type EvidenceItem = {
 id?: string;
 url: string;
 title?: string;
 publisher?: string;
 countryCode?: string;
 language?: string;
 publishedAt?: string;
 accessedAt: string;
 sourceType:
 | "official_government"
 | "election_commission"
 | "official_candidate"
 | "official_party"
 | "public_record"
 | "court_record"
 | "statistics_database"
 | "reputable_media"
 | "fact_checking_org"
 | "academic"
 | "ngo"
 | "social_media"
 | "other";
 relevantExcerpt?: string;
 relevanceScore: number;
 supportsClaim?: boolean;
 contradictsClaim?: boolean;
};

```

## 13.9 Research agent result

```

```ts id="jmcud" type ResearchAgentResult = { id: string; agentName: "RESEARCH_AGENT_A" |
"RESEARCH_AGENT_B"; claimId: string; promptVersion: string; modelName: string; rating: ClaimRating;
confidenceScore: number; summary: string; reasoningSummary: string; evidence: EvidenceItem[];
concerns: string[]; missingData: string[]; tokenUsage?: TokenUsage; createdAt: string; };

```

```

---

## 13.10 Audit agent result

```ts id="j205ff"
type AuditAgentResult = {
 id: string;
 claimId: string;

 agentAResultId: string;
 agentBResultId: string;

```

```

promptVersion: string;
modelName: string;

agreementLevel:
 | "HIGH"
 | "MEDIUM"
 | "LOW"
 | "CONFLICT";

finalRating: ClaimRating;
finalConfidenceScore: number;

publishStatus:
 | "APPROVED"
 | "APPROVED_WITH_WARNINGS"
 | "NEEDS_HUMAN_REVIEW"
 | "REJECTED";

auditSummary: string;
publicExplanation: string;
internalConcerns: string[];
requiredHumanReviewReason?: string;

tokenUsage?: TokenUsage;
createdAt: string;
};

```

## 13.11 Claim check

```

``ts id="iry0ud" type ClaimCheck = { id: string; claimId: string; candidateId?: string; partyId?: string;

```

```

finalRating: ClaimRating; confidenceScore: number;

```

```

shortExplanation: string; detailedExplanation: string;

```

```

evidenceUrls: string[]; conflictingEvidenceUrls?: string[];

```

```

tags: AnalysisTag[];

```

```

researchAgentAResultId: string; researchAgentBResultId: string; auditResultId: string;

```

```

auditStatus: | "PASSED" | "PASSED_WITH_WARNINGS" | "FAILED_NEEDS_REVIEW" |
"INSUFFICIENT_EVIDENCE";

```

```

traceId: string;

```

```

createdAt: string; updatedAt: string; };

```

---

## 13.12 Candidate verdict

```
``ts id="xvddnx"
type CandidateVerdict = {
 id: string;
 candidateId: string;
 electionId: string;
 verdict: PizditVerdict;
 confidenceScore: number;
 summary: string;
 topConcerns: string[];
 reasonableProposals: string[];
 tags: AnalysisTag[];
 checkedClaimsCount: number;
 problematicClaimsCount: number;
 lastCalculatedAt: string;
};
```

---

## 13.13 Token usage

```
``ts id="yr9gia" type TokenUsage = { inputTokens: number; outputTokens: number; totalTokens:
number; estimatedCostUsd?: number; };
```

---

## 13.14 Claim check trace

```
``ts id="l3gxzz"
type ClaimCheckTrace = {
 id: string;
 claimId: string;
 agentAResultId: string;
 agentBResultId: string;
 auditResultId: string;
 sourceSnapshotIds: string[];
 modelNames: string[];
 promptVersions: string[];
 tokenUsage: TokenUsage;
 createdAt: string;
};
```

## 14. Country Adapter Architecture

Each country should have its own adapter for election data, district mapping, candidate discovery, and source ingestion.

```
``ts id="fygtdg" interface CountryAdapter { countryCode: string;

getUpcomingElections(input: { regionName?: string; municipalityName?: string; districtName?: string; }):
Promise<Election[]>;

getDistrictForAddress(input: { address?: string; latitude?: number; longitude?: number; }):
Promise<ElectoralDistrict | null>;

getCandidatesForElection(electionId: string): Promise<Candidate[]>;

getOfficialSourcesForCandidate(candidateId: string): Promise<CandidateSource[]>;

getOfficialSourcesForParty?(partyId: string): Promise<CandidateSource[]>;

normalizeAddress?(address: string): Promise<string>;

getPollingPlaceUrl?(input: { address?: string; districtId?: string; }): Promise<string | null>; }
```

The app should include a generic fallback adapter for countries where structured data is not yet available.

Adapter status levels:

```
``ts id="m6o5e5"
type AdapterStatus =
 | "NOT_STARTED"
 | "PLANNED"
 | "IN_PROGRESS"
 | "BETA"
 | "SUPPORTED"
 | "DEPRECATED";
```

---

## 15. AI Fact-Checking Architecture

The system must use a three-stage AI pipeline.

```
``text id="erm37k" Political claim ↓ Research Agent A ↓ Research Agent B ↓ Audit Agent ↓ Publish /
Human review / Reject
```

Research Agent A and Research Agent B must work independently.

The Audit Agent receives both outputs and decides whether the result is safe and reliable enough to publish.

---

#### ## 16. Research Agent A

Research Agent A independently verifies the claim.

Responsibilities:

- identify the factual core of the claim;
- search for official sources;
- search for public records;
- search for statistics;
- search for reliable media coverage;
- identify supporting evidence;
- identify contradicting evidence;
- detect missing context;
- detect jurisdiction issues;
- rate the claim;
- provide source URLs;
- provide confidence score;
- provide reasoning summary.

Research Agent A must not see Agent B's result.

---

#### ## 17. Research Agent B

Research Agent B performs the same verification independently.

Responsibilities:

- verify the same claim from another angle;
- prioritize primary sources where possible;
- look for both supporting and contradicting evidence;
- check dates and jurisdiction;
- check whether the candidate has legal authority to implement the promise;
- detect statistical manipulation;
- detect missing budget, timeline, or implementation plan;
- provide independent rating;
- provide independent confidence score;
- provide independent source list.

Research Agent B must not see Agent A's result.

---

#### ## 18. Audit Agent

The Audit Agent is stricter than both research agents.

It receives:

- original claim;
- candidate context;
- Research Agent A result;
- Research Agent B result;
- source lists from both agents;
- evidence excerpts;
- confidence scores.

Responsibilities:

- compare Agent A and Agent B conclusions;
- detect disagreement;
- detect weak evidence;
- detect hallucinated or irrelevant sources;
- check whether sources actually support the conclusions;
- lower confidence if evidence is incomplete;
- reject results that rely on low-quality sources;
- send risky cases to human review;
- produce the final public explanation;
- decide publish status.

The Audit Agent must not automatically average the two research results. It must evaluate the quality of evidence.

---

## ## 19. AI Pipeline Logic

### ## 19.1 Source ingestion

```text id="l3ax5v"

Input:

Candidate websites, party programs, election commission records, government records, public speeches, interviews, voting records, public statistics, reputable media.

Pipeline:

1. Collect source URLs.
2. Download source content where legally allowed.
3. Store source snapshots.
4. Split documents into chunks.
5. Store chunks with metadata.
6. Generate embeddings if needed.
7. Mark source type and reliability level.
8. Make source chunks available for claim extraction and fact-checking.

19.2 Claim extraction

```text id="h93ix4" Input: Candidate profile, program, public statements, media links, official sources.

Pipeline: 1. Extract checkable claims. 2. Ignore vague slogans unless they contain factual or policy-checkable content. 3. Deduplicate similar claims. 4. Classify claim type. 5. Preserve original language. 6. Add translation when useful. 7. Store claim with source URL and context.

---

### ## 19.3 Claim processing

```text id="mzn16y"

Input:

Political claim + relevant source context pack.

Pipeline:

1. Retrieve relevant source chunks.
2. Build compact context pack.
3. Send claim and context pack to Research Agent A.
4. Send claim and context pack to Research Agent B.
5. Store raw results.
6. Send both outputs to Audit Agent.
7. If audit approves, create public ClaimCheck.
8. If audit fails or confidence is too low, send to Human Review Queue.
9. Aggregate approved claim checks into candidate-level verdict.

19.4 Candidate verdict aggregation

The candidate verdict must not be generated directly by a single AI call.

It must be computed from approved claim checks, tags, confidence scores, and audit statuses.

Example logic:

```
```ts id="r8an44" function calculateCandidateVerdict(claimChecks: ClaimCheck[]): PizditVerdict { const approvedChecks = claimChecks.filter( check => check.auditStatus === "PASSED" || check.auditStatus === "PASSED_WITH_WARNINGS" );
```

```
if (approvedChecks.length < 3) { return "INSUFFICIENT_DATA"; }
```

```
const problematic = approvedChecks.filter(check => ["MISLEADING", "UNSUPPORTED", "FALSE"].includes(check.finalRating));
```

```
const mixed = approvedChecks.filter(check => check.finalRating === "MIXED");
```

```
const problematicRatio = problematic.length / approvedChecks.length;

if (problematicRatio >= 0.6) return "YES_OFTEN"; if (problematicRatio >= 0.35) return "LIKELY_YES"; if
(problematicRatio >= 0.2 || mixed.length >= 2) return "MIXED";

return "LIKELY_NO"; }
```

This logic should be configurable in the admin panel or environment config.

---

## ## 20. Token and Cost Strategy

The system should be designed to keep token usage low.

The main cost driver is not displaying candidate statistics. The expensive part is initial source processing and claim verification.

Cheap operations:

- showing candidate list;
- showing existing verdicts;
- filtering by address;
- opening candidate pages;
- opening claim pages;
- recalculating candidate verdict from existing claim checks;
- reading cached database data.

Expensive operations:

- reading long candidate programs;
- parsing many interviews;
- processing long public speeches;
- analyzing legislative voting history;
- searching and collecting external sources;
- running two research agents and one audit agent per claim.

---

### ## 20.1 No repeated full-context analysis

The system must not send full candidate data to AI repeatedly.

Instead:

- ```
```text id="ixpdfc"
1. Collect sources once.
2. Chunk sources.
3. Store chunks.
4. Extract claims once.
```

5. Retrieve only relevant chunks per claim.
6. Run Agent A.
7. Run Agent B.
8. Run Audit Agent.
9. Store final ClaimCheck.
10. Public app reads stored result.

---

## 20.2 Approximate token expectations

Rough candidate-level estimates:

```text id="ans3pq" Candidate source ingestion: 10,000–80,000 input tokens

Claim extraction: 5,000–30,000 input tokens 1,000–5,000 output tokens

Per claim fact-check: Agent A: 3,000–12,000 input + 500–2,000 output Agent B: 3,000–12,000 input + 500–2,000 output Audit: 2,000–8,000 input + 500–1,500 output

For a candidate with 10 important claims:

```text id="q0eb09"  
Approximate total:  
80,000–300,000 input tokens  
10,000–40,000 output tokens

This should be manageable if the system uses:

- cheap models for extraction;
- stronger models only for audit or high-risk cases;
- caching;
- source chunk retrieval;
- batch processing;
- human review for edge cases.

---

## 20.3 Cost control rules

1. No live public AI calls.
2. Cache every source snapshot.
3. Cache every extracted claim.
4. Cache every claim check.
5. Store prompt version and model version.
6. Re-run checks only when sources, prompts, models, or methodology change.
7. Use smaller models for low-risk extraction tasks.
8. Use stronger models for audit and high-risk claims.
9. Use retrieval to send only relevant context.

10. Track token usage per pipeline run.

---

## 21. Source Quality Rules

Source priority, from strongest to weakest:

1. Election commission data
2. Government records
3. Parliamentary, municipal, or legislative voting records
4. Official candidate programs
5. Official party programs
6. Public statistics databases
7. Court records
8. Reputable media
9. Independent fact-checking organizations
10. Academic or NGO reports
11. Verified social media accounts of candidates or parties
12. Other sources

Weak sources must not be used as primary evidence unless clearly marked.

Weak sources include:

- anonymous blogs;
  - partisan channels without evidence;
  - unsourced screenshots;
  - edited clips without full context;
  - anonymous social media posts;
  - campaign attack ads without supporting documentation.
- 

## 22. Human Review Triggers

A claim must be sent to human review if:

- Research Agent A and Research Agent B strongly disagree;
- evidence is weak;
- evidence is unavailable;
- source quality is low;
- the claim involves serious allegations;
- the claim involves criminal accusations;
- the claim involves possible defamation risk;
- the claim involves sensitive personal data;
- the Audit Agent detects hallucinated sources;
- the Audit Agent confidence is below threshold;
- the claim could materially affect a candidate's reputation;
- the claim requires local legal or political context that AI may not handle reliably.

Default threshold:

```
```ts id="oz88a3" const MIN_CONFIDENCE_TO_PUBLISH = 0.72;
```

If confidence is below threshold:

```
```ts id="4k4qs7"
publishStatus = "NEEDS_HUMAN_REVIEW";
```

---

## 23. Open-Source Strategy

The project should be open-source to build public trust.

Recommended repository structure:

```
```text id="owgsza" / apps/ web/ admin/ packages/ core/ db/ ai/ adapters/ ui/ country-adapters/ pt/ us/
ge/ uk/ generic/ methodology/ source-quality-rules.md claim-rating-rules.md audit-rules.md candidate-
verdict-formula.md human-review-rules.md prompts/ research-agent-a.md research-agent-b.md audit-
agent.md claim-extraction.md evaluation/ test-claims.json expected-ratings.json source-fixtures.json
docs/ architecture.md contributing.md adapter-guide.md methodology.md privacy.md
```

```
---
```

23.1 What should be open

Open-source repository should include:

- frontend code;
- backend code;
- database schema;
- type definitions;
- country adapter interfaces;
- example adapters;
- AI prompts;
- methodology;
- scoring logic;
- audit logic;
- source quality rules;
- evaluation tests;
- documentation.

```
---
```

23.2 What should not be public

The public repository must not include:

- API keys;
- private admin credentials;

- private source access tokens;
- production secrets;
- sensitive moderation notes;
- unverified allegations not approved for publication;
- private user data;
- raw address logs;
- legally sensitive unpublished reviews.

Open-source does not mean exposing unsafe or private production data.

23.3 Contributor model

Contributors should be able to help with:

- country adapters;
- source mapping;
- translation;
- UI improvements;
- methodology review;
- prompt improvements;
- test cases;
- documentation;
- data quality checks.

High-risk contributions, especially political analysis changes, must require review.

24. User Interface

24.1 Screen: Address Input

Purpose:

Help the user find their relevant elections without registration.

Elements:

- app logo;
- product name: `Pizdit?`;
- subtitle;
- address input;
- manual country/region selector;
- privacy note.

Example copy:

```
```text id="xlka3z"
```

See what politicians are saying before you vote.

Enter your address.

No account needed.

Privacy copy:

```text id="2c9871" We use your address only to find your election district.

24.2 Screen: Election Overview

Elements:

- election name;
- election date;
- election level;
- country;
- region;
- district;
- polling place link, if available;
- list of candidates or parties.

Example:

```text id="scf16e"

Municipal Council Election

Porto, Portugal

12 October 2026

Your ballot:

Candidate A

Candidate B

Candidate C

Party D

---

## 24.3 Screen: Candidate List

Candidate card elements:

- name;
- party;
- photo, optional;
- verdict badge;
- top 3 tags;
- one-line summary;

- CTA button.

Example:

```text id="v5xpq3" Anna Silva Independent

Pizdit? Likely yes

Flags: - Promises without budget - Uses misleading statistics - No implementation plan

[View analysis](#)

24.4 Screen: Candidate Detail

Sections:

1. Candidate identity
2. Verdict summary
3. Main concerns
4. Reasonable proposals
5. Checked claims
6. Program analysis
7. Source list
8. Audit trail summary
9. Last updated timestamp

Example verdict summary:

```text id="u2vtmr"

Likely yes

Several key claims lack clear evidence, budget details, or implementation mechanisms. Two claims contradict previous public positions.

---

## 24.5 Screen: Claim Detail

Elements:

- claim text;
- claim source;
- claim date;
- rating;
- confidence score;
- explanation;
- evidence;
- conflicting evidence;

- audit status;
- human review status, if applicable;
- prompt version;
- model version;
- source snapshot date.

Example:

```text id="gnmy7a" Claim: "We will reduce housing prices by 50% in one year."```

Rating: Unsupported

Why: The candidate does not provide a budget, legal mechanism, implementation timeline, or market intervention plan. Available housing data does not support the feasibility of this scale of reduction within one year.

Audit: Passed with warnings

Reason: Both research agents rated the claim as unsupported. However, current-year housing data is incomplete.

25. Admin Panel

The admin panel must be protected.

Admin panel sections:

1. Countries
2. Country adapters
3. Elections
4. Districts
5. Parties
6. Candidates
7. Sources
8. Claims
9. AI pipeline runs
10. Research Agent A output
11. Research Agent B output
12. Audit Agent output
13. Human review queue
14. Manual corrections
15. Publish/unpublish controls
16. Token usage dashboard
17. Prompt version management
18. Model configuration

Manual corrections must always be logged.

Manual override data:

```
``ts id="g3zvg8"
type ManualOverride = {
  id: string;
  entityType: "claim_check" | "candidate_verdict" | "source" | "candidate" |
"election";
  entityId: string;
  previousValue: unknown;
  newValue: unknown;
  reason: string;
  reviewerId: string;
  createdAt: string;
};
```

26. Recommended Tech Stack

Recommended stack:

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Supabase
- PostgreSQL
- Supabase Storage
- Vercel
- Geocoding provider
- AI provider abstraction layer
- Queue or background job system
- Vector storage for source chunks, if needed

The system must support replacing AI providers later.

Do not hardcode one AI model provider into the business logic.

Create an interface:

```
``ts id="q5g6ec" interface AIProvider { runResearchAgent(input: ResearchAgentInput):
Promise<ResearchAgentResult>; runAuditAgent(input: AuditAgentInput): Promise<AuditAgentResult>;
extractClaims(input: ClaimExtractionInput): Promise<PoliticalClaim[]>; }
```

27. Database Tables

Recommended tables:

```
```text id="jqc76g"
countries
country_adapters
elections
electoral_districts
parties
candidates
candidate_sources
source_snapshots
source_chunks
political_claims
claim_checks
research_agent_results
audit_agent_results
evidence_items
candidate_verdicts
geocoding_cache
pipeline_runs
human_review_queue
manual_overrides
prompt_versions
model_configs
claim_check_traces
token_usage_logs
```

---

## 28. Supabase Schema Draft

```
```sql id="o5tn6s" create table countries ( id uuid primary key default gen_random_uuid(), iso_code text
unique not null, name text not null, default_language text not null, supported_languages text[] default
'{}', election_system_notes text, legal_risk_notes text, official_election_source_urls text[] default '{}',
adapter_status text not null default 'NOT_STARTED', created_at timestamptz default now(), updated_at
timestamptz default now() );
```

```
create table elections ( id uuid primary key default gen_random_uuid(), country_code text not null, name
text not null, region_name text, municipality_name text, district_name text, election_level text not null,
ballot_type text not null default 'unknown', election_date date not null, registration_deadline date,
polling_place_url text, source_urls text[] default '{}', created_at timestamptz default now(), updated_at
timestamptz default now() );
```

```
create table electoral_districts ( id uuid primary key default gen_random_uuid(), country_code text not
null, region_name text, municipality_name text, district_name text not null, district_code text,
boundary_geojson jsonb, source_urls text[] default '{}', created_at timestamptz default now(),
updated_at timestamptz default now() );
```

```
create table parties ( id uuid primary key default gen_random_uuid(), country_code text not null, name
text not null, short_name text, official_website_url text, social_urls text[] default '{}', ideology_description
text, source_urls text[] default '{}', created_at timestamptz default now(), updated_at timestamptz
default now() );
```

```
create table candidates ( id uuid primary key default gen_random_uuid(), election_id uuid references elections(id) on delete cascade, country_code text not null, full_name text not null, party_name text, party_id uuid references parties(id), is_independent boolean default false, photo_url text, official_website_url text, social_urls text[] default '{}', short_bio text, current_role text, previous_roles text[] default '{}', district_name text, source_language text, created_at timestampz default now(), updated_at timestampz default now() );
```

```
create table candidate_sources ( id uuid primary key default gen_random_uuid(), candidate_id uuid references candidates(id) on delete cascade, url text not null, title text, publisher text, country_code text not null, language text, source_type text not null, reliability_level text not null, added_at timestampz default now() );
```

```
create table source_snapshots ( id uuid primary key default gen_random_uuid(), url text not null, title text, publisher text, country_code text, language text, content_hash text, raw_text text, raw_html text, captured_at timestampz default now(), created_at timestampz default now() );
```

```
create table source_chunks ( id uuid primary key default gen_random_uuid(), source_snapshot_id uuid references source_snapshots(id) on delete cascade, chunk_index integer not null, chunk_text text not null, token_count integer, embedding vector, metadata jsonb, created_at timestampz default now() );
```

```
create table political_claims ( id uuid primary key default gen_random_uuid(), candidate_id uuid references candidates(id) on delete cascade, party_id uuid references parties(id) on delete cascade, election_id uuid references elections(id) on delete cascade, country_code text not null, claim_text text not null, original_language text not null, translated_text text, claim_type text not null, claim_source_url text not null, claim_source_title text, claim_date date, context text, jurisdiction_level text, created_at timestampz default now(), updated_at timestampz default now() );
```

```
create table research_agent_results ( id uuid primary key default gen_random_uuid(), claim_id uuid references political_claims(id) on delete cascade, agent_name text not null, prompt_version text not null, model_name text not null, rating text not null, confidence_score numeric not null, summary text not null, reasoning_summary text not null, concerns text[] default '{}', missing_data text[] default '{}', raw_output jsonb, input_tokens integer, output_tokens integer, estimated_cost_usd numeric, created_at timestampz default now() );
```

```
create table evidence_items ( id uuid primary key default gen_random_uuid(), research_agent_result_id uuid references research_agent_results(id) on delete cascade, url text not null, title text, publisher text, country_code text, language text, published_at date, accessed_at timestampz default now(), source_type text not null, relevant_excerpt text, relevance_score numeric not null, supports_claim boolean, contradicts_claim boolean, created_at timestampz default now() );
```

```
create table audit_agent_results ( id uuid primary key default gen_random_uuid(), claim_id uuid references political_claims(id) on delete cascade, agent_a_result_id uuid references research_agent_results(id), agent_b_result_id uuid references research_agent_results(id), prompt_version text not null, model_name text not null, agreement_level text not null, final_rating text not null, final_confidence_score numeric not null, publish_status text not null, audit_summary text not null, public_explanation text not null, internal_concerns text[] default '{}', required_human_review_reason text, raw_output jsonb, input_tokens integer, output_tokens integer, estimated_cost_usd numeric, created_at timestampz default now() );
```

```
create table claim_check_traces ( id uuid primary key default gen_random_uuid(), claim_id uuid
references political_claims(id) on delete cascade, agent_a_result_id uuid references
research_agent_results(id), agent_b_result_id uuid references research_agent_results(id), audit_result_id
uuid references audit_agent_results(id), source_snapshot_ids uuid[] default '{}', model_names text[]
default '{}', prompt_versions text[] default '{}', input_tokens integer default 0, output_tokens integer
default 0, estimated_cost_usd numeric, created_at timestamptz default now() );
```

```
create table claim_checks ( id uuid primary key default gen_random_uuid(), claim_id uuid references
political_claims(id) on delete cascade, candidate_id uuid references candidates(id) on delete cascade,
party_id uuid references parties(id) on delete cascade, final_rating text not null, confidence_score
numeric not null, short_explanation text not null, detailed_explanation text not null, evidence_urls text[]
default '{}', conflicting_evidence_urls text[] default '{}', tags text[] default '{}', research_agent_a_result_id
uuid references research_agent_results(id), research_agent_b_result_id uuid references
research_agent_results(id), audit_result_id uuid references audit_agent_results(id), trace_id uuid
references claim_check_traces(id), audit_status text not null, created_at timestamptz default now(),
updated_at timestamptz default now() );
```

```
create table candidate_verdicts ( id uuid primary key default gen_random_uuid(), candidate_id uuid
references candidates(id) on delete cascade, election_id uuid references elections(id) on delete cascade,
verdict text not null, confidence_score numeric not null, summary text not null, top_concerns text[]
default '{}', reasonable_proposals text[] default '{}', tags text[] default '{}', checked_claims_count integer
default 0, problematic_claims_count integer default 0, last_calculated_at timestamptz default now() );
```

```
create table geocoding_cache ( id uuid primary key default gen_random_uuid(),
normalized_address_hash text unique not null, country_code text not null, region_name text,
municipality_name text, district_name text, latitude numeric, longitude numeric, provider text,
raw_result jsonb, created_at timestamptz default now() );
```

```
create table pipeline_runs ( id uuid primary key default gen_random_uuid(), entity_type text not null,
entity_id uuid not null, pipeline_type text not null, status text not null default 'pending', error_message
text, started_at timestamptz, finished_at timestamptz, created_at timestamptz default now() );
```

```
create table human_review_queue ( id uuid primary key default gen_random_uuid(), claim_id uuid
references political_claims(id) on delete cascade, audit_result_id uuid references audit_agent_results(id),
reason text not null, status text default 'pending', reviewer_notes text, created_at timestamptz default
now(), updated_at timestamptz default now() );
```

```
create table manual_overrides ( id uuid primary key default gen_random_uuid(), entity_type text not
null, entity_id uuid not null, previous_value jsonb, new_value jsonb not null, reason text not null,
reviewer_id text, created_at timestamptz default now() );
```

```
create table prompt_versions ( id uuid primary key default gen_random_uuid(), name text not null,
version text not null, prompt_text text not null, created_at timestamptz default now() );
```

```
create table model_configs ( id uuid primary key default gen_random_uuid(), name text not null,
provider text not null, model_name text not null, role text not null, input_cost_per_million_tokens
numeric, output_cost_per_million_tokens numeric, is_active boolean default true, created_at
timestamptz default now() );
```

```
create table token_usage_logs ( id uuid primary key default gen_random_uuid(), pipeline_run_id uuid
references pipeline_runs(id), entity_type text not null, entity_id uuid not null, model_name text not null,
prompt_version text, input_tokens integer not null, output_tokens integer not null, estimated_cost_usd
numeric, created_at timestampz default now() );
```

Note: if using Supabase Postgres vector embeddings, enable the vector extension before using the `embedding vector` column.

29. API Routes

Public API routes:

```
```text id="t07gd1"
POST /api/geocode
GET /api/countries
GET /api/countries/:countryCode
GET /api/elections?countryCode=&districtId=
GET /api/elections/:id
GET /api/elections/:id/candidates
GET /api/candidates/:id
GET /api/candidates/:id/claims
GET /api/claims/:id
```

Admin API routes:

```
```text id="cyayt8" POST /api/admin/countries PATCH /api/admin/countries/:id
```

```
POST /api/admin/elections PATCH /api/admin/elections/:id DELETE /api/admin/elections/:id
```

```
POST /api/admin/candidates PATCH /api/admin/candidates/:id DELETE /api/admin/candidates/:id
```

```
POST /api/admin/sources PATCH /api/admin/sources/:id DELETE /api/admin/sources/:id
```

```
POST /api/admin/pipeline/ingest-sources POST /api/admin/pipeline/extract-claims POST /api/admin/
pipeline/check-claim POST /api/admin/pipeline/recalculate-verdict
```

```
GET /api/admin/review POST /api/admin/review/:id/approve POST /api/admin/review/:id/reject POST /
api/admin/review/:id/request-changes
```

```
GET /api/admin/token-usage GET /api/admin/prompt-versions POST /api/admin/prompt-versions
```

30. Initial App Routes

Create these routes first:

```
```text id="bkzdk8"
/
/countries
/elections/[electionId]
/candidates/[candidateId]
/claims/[claimId]
/admin
/admin/countries
/admin/elections
/admin/candidates
/admin/claims
/admin/review
/admin/token-usage
/admin/prompts
```

---

## 31. Core Components

Create these reusable components:

```
```text id="w29byd" AddressInput CountrySelector RegionSelector ElectionCard CandidateCard
VerdictBadge AnalysisTagList ClaimCheckCard EvidenceList AuditStatusBadge
CandidateVerdictSummary SourceQualityBadge HumanReviewStatus TokenUsageBadge
PromptVersionBadge CountryCoverageBadge
```

```
---
```

32. AI Prompt Drafts

32.1 Claim extraction prompt

```
```text id="ileztx"
```

You are a claim extraction agent for a political fact-checking system.

Your task is to extract checkable political claims from the provided source text.

Rules:

- Extract only claims that can be checked with evidence.
- Preserve the original wording when possible.
- Keep the original language.
- Add an English translation if needed.
- Ignore vague slogans unless they imply a factual or policy-checkable statement.
- Classify each claim by type.
- Include source URL, source title, date if known, and surrounding context.

- Do not rate the claim.
- Return structured JSON only.

---

## 32.2 Research Agent A prompt

```text id="plm156" You are Research Agent A for a political fact-checking system.

Your task is to independently verify the provided political claim.

Rules: - Use only reliable sources. - Prefer official government, election, statistical, legal, parliamentary, municipal, and primary sources. - Do not assume the claim is false. - Do not assume the claim is true. - Identify whether the claim is factual, misleading, unsupported, false, mixed, or unverifiable. - Provide evidence URLs. - Explain what is confirmed, what is contradicted, and what remains unknown. - Check whether the candidate has legal authority to implement the promise. - Check whether the claim includes a budget, timeline, and implementation mechanism where relevant. - Do not produce a publishable verdict if evidence is weak. - Return structured JSON only.

32.3 Research Agent B prompt

```text id="ihrl8o"

You are Research Agent B for a political fact-checking system.

Your task is to verify the same claim independently from another angle.

Rules:

- Do not rely on assumptions.
- Look for primary sources first.
- Try to find evidence that could both support and contradict the claim.
- Be careful with statistics, dates, jurisdiction, and legal authority.
- Check whether the politician has the actual power to implement the promise.
- Check whether comparable policies have been implemented elsewhere, if relevant.
- Mark the claim as unverifiable if reliable evidence is insufficient.
- Include missing information and uncertainty.
- Return structured JSON only.

---

## 32.4 Audit Agent prompt

```text id="mrbn7p" You are the Audit Agent for a political fact-checking system.

You receive: - the original political claim; - candidate context; - Research Agent A result; - Research Agent B result; - their source lists; - their evidence excerpts.

Your job: - compare both research results; - check if their sources actually support their conclusions; - detect hallucinated, weak, outdated, or irrelevant sources; - detect disagreement between agents; - decide whether the claim check can be published; - produce the final rating only if evidence is strong enough; - lower confidence when evidence is incomplete; - send to human review if legal, factual, or evidentiary risk is high; - be stricter than both research agents.

Return structured JSON only.

33. Privacy

The app should collect as little user data as possible.

Address handling rules:

- Use the address only to determine the election district.
- Do not create public user profiles.
- Do not store user search history in MVP.
- Store geocoding cache using normalized address hashes, not raw personally identifiable address records, where possible.
- Do not sell or share user location data.
- Provide manual region selection as an alternative to address input.
- Do not use voter-facing address data for analytics beyond anonymous aggregate diagnostics.

Privacy copy:

```text id="sfrlco"

We use your address only to find your election district. No account needed.

---

## 34. Legal and Safety Rules

The product must minimize defamation and misinformation risk.

### Avoid direct personal accusations

Do not write:

```text id="ptsu2p" This candidate is a criminal.

Write:

```text id="4p85s4"

Court records show an ongoing case related to [topic]. The system does not make a conclusion about guilt.

## Avoid unsupported corruption claims

Do not write:

```text id="n8gugb" This candidate is corrupt.

Write:

```text id="jr3c0s"  
Public reporting from [source] describes allegations related to [topic]. The available evidence does not establish a final legal finding.

## Avoid unsupported lying claims

Do not write:

```text id="pria4f" This candidate lies.

Write:

```text id="0ed51f"  
This specific claim is not supported by the available evidence.

## Use claim-level analysis

The system should evaluate claims, statements, promises, and records. Candidate-level verdicts must be derived from checked claim-level results.

---

## 35. Success Metrics

### 35.1 Product metrics

- address-to-election success rate;
  - manual region fallback usage;
  - percentage of users who open candidate cards;
  - percentage of users who open claim details;
  - percentage of users who open source links;
  - number of supported countries;
  - number of supported elections;
  - number of candidates with completed analysis;
  - number of claims checked;
  - number of published claim checks.
-

## 35.2 Quality metrics

- average number of sources per claim;
  - Agent A/B agreement rate;
  - Audit Agent rejection rate;
  - human review rate;
  - percentage of claims marked insufficient data;
  - percentage of corrected claim checks after review;
  - average confidence score of published checks;
  - percentage of checks with primary sources;
  - average token cost per claim check;
  - average token cost per candidate;
  - percentage of stale checks requiring revalidation.
- 

## 36. Global Build Order

### Phase 1: Static global prototype

- Create public app shell.
  - Create address input UI.
  - Create country selector.
  - Create hardcoded election overview.
  - Create hardcoded candidate list.
  - Create hardcoded candidate detail.
  - Create hardcoded claim detail.
- 

### Phase 2: Core database

- Add Supabase.
  - Add country tables.
  - Add election tables.
  - Add district tables.
  - Add party tables.
  - Add candidate tables.
  - Add claim tables.
  - Add verdict tables.
  - Add source tables.
  - Add trace and token usage tables.
- 

### Phase 3: Country adapter layer

- Add country adapter interface.
- Add generic fallback adapter.
- Add first real country adapter.
- Add adapter status model.
- Add adapter guide documentation.

---

## Phase 4: Address flow

- Add geocoding.
  - Add district matching.
  - Add election lookup.
  - Add manual fallback.
  - Add country coverage states.
- 

## Phase 5: Admin panel

- Add country management.
  - Add election management.
  - Add party management.
  - Add candidate management.
  - Add source management.
  - Add claim management.
  - Add prompt management.
  - Add token usage dashboard.
- 

## Phase 6: Source ingestion

- Add source snapshot storage.
  - Add source chunking.
  - Add source metadata.
  - Add reliability labels.
  - Add optional embeddings.
- 

## Phase 7: AI pipeline

- Add claim extraction.
  - Add Research Agent A.
  - Add Research Agent B.
  - Add Audit Agent.
  - Store raw outputs.
  - Store evidence items.
  - Store claim checks.
  - Store trace records.
  - Store token usage.
- 

## Phase 8: Human review

- Add review queue.
- Add approve/reject flow.

- Add manual override logging.
  - Add publish/unpublish controls.
- 

## Phase 9: Public MVP

- Launch global-capable app.
  - Start with limited country coverage.
  - Add public coverage map.
  - Add real candidates.
  - Add real source documents.
  - Publish audited claim checks only.
- 

## 37. Cursor Implementation Instructions

Build the app as a **Next.js + TypeScript + Supabase** project.

Use:

- Next.js App Router
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Supabase
- PostgreSQL
- Vercel deployment
- queue/background job system for AI pipeline
- optional vector search for source chunks

Prioritize:

1. clean global data models;
2. strict TypeScript types;
3. simple public UX;
4. reusable UI components;
5. source-backed claim analysis;
6. structured AI outputs;
7. auditability;
8. human review support;
9. no public authentication;
10. protected admin routes;
11. modular country adapters;
12. no live AI calls in public voter-facing flow;
13. token and cost tracking;
14. open-source methodology.

Public users should never need to log in.

Admin users may need authentication.

Start with mocked data. Then connect Supabase.

Do not implement:

- public comments;
- social features;
- accounts for voters;
- recommendation algorithms;
- “vote for this candidate” features;
- anonymous accusation submission;
- public candidate rating by users;
- public live AI analysis.

The product should feel extremely simple on the surface and deeply rigorous inside.

---

## 38. Naming Notes

The public product name can remain:

```text id="b92v97" Pizdit?

However, internal technical naming should avoid legally risky terms like:

```
```text id="qf5ujz"
lie_score
liar_rating
corruption_score
```

Prefer:

```text id="o474a0" verification\_risk evidence\_status claim\_integrity pizdit\_verdict claim\_rating
audit_status

This keeps the product expressive while reducing legal and reputational risk in the underlying system.

39. Core Architectural Rule

The most important technical rule:

```
```text id="uwpcag"
No live AI calls in the voter-facing flow.
```

The second most important rule:

``text id="emcslk" Every public claim check must be traceable to sources, prompts, model versions, and audit results.

The third most important rule:

```text id="h1ib40"

Candidate-level verdicts must be computed from audited claim-level checks, not generated directly by a single AI model.